BEDROCK ARCHITECTURE

Bedrock ELF ABI

ELF Object, Linking, and Loading ABI

Draft

CONTENTS

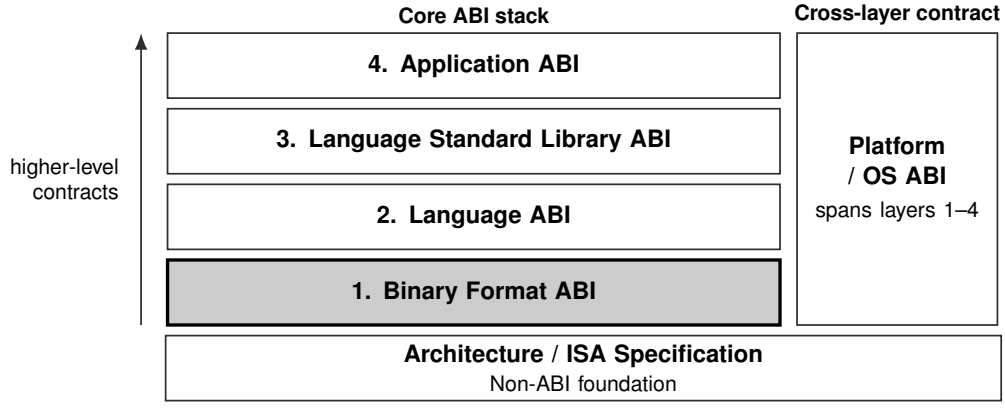
LIST OF TABLES

LIST OF FIGURES

1 SCOPE AND CONFORMANCE

This document defines the Bedrock binary-format contract for ELF objects, linking, relocation, loading, dynamic linking, TLS metadata, and code models. It depends on the Bedrock ISA Specification and the active platform/runtime environment ABI. It does not define a language calling convention or an application interface.

1.1 Contract Position



Current document: Bedrock ELF ABI. The shaded block identifies its primary contract position. The Platform / OS ABI spans and constrains the four core ABI layers; the ISA specification is their non-ABI architectural foundation.

1.2 Profile

Profile: `bedrock-elf`

Optional profile: `bedrock-elf-far`

2 TERMINOLOGY

These terms are used by the Bedrock ELF ABI. ISA-level address, register, and effective-address terms keep the meanings defined by the architecture manual.

ELF ABI The ELF, relocation, dynamic-linking, TLS, and code-model contract that is shared by language bindings and operating-system profiles.

platform/runtime environment ABI
The lower-layer execution-environment contract that defines process entry, system-call transitions, thread and TLS setup, loader handoff, signal or exception delivery, and platform segment policy.

pre-segment virtual address
The address value used by ELF symbols, relocations, ordinary function addresses, and ordinary data pointers before segment pre-translation. It is numerically identical to the resulting linear address in the disabled and bounds-only contexts required by the Bedrock C ABI.

load base The runtime address bias applied to a loaded ET_DYN object before its symbol values and relative relocations are interpreted.

ABI-visible linkage domain
The region in which ordinary ELF code addresses are near-call entry points and do not carry an explicit CS value.

near-call entry address
A pre-segment code address that can be reached with ordinary CALL and returned from with ordinary RET inside the current linkage domain.

veneer A near-call entry point supplied by a linker, loader, runtime, or OS ABI to hide a more complex transfer sequence from ordinary ABI callers.

code model A set of placement and relocation assumptions used by code generation to choose direct displacement forms, GOT/PLT forms, or full-width address materialization.

segment domain A linker-visible data-address domain whose symbols share one final bounds-only segment register image and use canonical pre-segment virtual addresses as their ABI-visible address values.

far pointer object A 16-byte object containing a 64-bit canonical pre-segment virtual address at byte zero and a 64-bit canonical disabled or bounds-only segment image at byte eight. Data and far function pointers use the same object layout but distinct language types.

3 ELF OBJECT FORMAT

Bedrock objects use the 64-bit little-endian ELF format and machine identifier `EM_BEDROCK` (`0xffb0`). A conforming producer may emit relocatable objects, executable images, or shared/position-independent images using `ET_REL`, `ET_EXEC`, or `ET_DYN`.

All relocation sections use `Elf64_Rela` entries with explicit addends and the `.rela` section-name stem. Ordinary symbol and string tables use `.symtab` and `.strtab`. Architecture attributes use `.note.bedrock`. The baseline value of `e_flags` is zero; a consumer must reject an unknown nonzero value rather than silently treating it as the baseline profile.

3.1 ELF Identification Bytes

Table 3-1. ELF e_ident Values

e_ident Field	Value
`EI_MAG`	`0x7f 'E' 'L' 'F'`
`EI_CLASS`	`ELFCLASS64`
`EI_DATA`	`ELFDATA2LSB`
`EI_VERSION`	`EV_CURRENT`
`EI_OSABI`	`ELFOSABI_NONE`
`EI_ABIVERSION`	`0`

3.2 ELF Header Sizes

Table 3-2. ELF Header Size Values

Header Field	Bytes
`e_ehsize`	64
`e_phentsize`	56
`e_shentsize`	64

3.3 Entry Point Semantics

Table 3-3. ELF Entry Point Rules

File Type	e_entry Meaning
`ET_REL`	0
`ET_EXEC`	absolute pre-segment virtual byte address
`ET_DYN (executable)`	load-bias-relative pre-segment virtual byte address; runtime entry is load base + e entry
`ET_DYN (shared object without entry)`	0

3.4 Program Header Policy

The standard `PT_LOAD`, `PT_DYNAMIC`, `PT_INTERP`, `PT_NOTE`, and `PT_TLS` program headers retain their ordinary ELF meanings. The far profile additionally defines `PT_BEDROCK_SEGDOM`; Section 6 gives its complete contract.

Program permission bits use the standard values PF_X=1, PF_W=2, and PF_R=4. The processor-specific PF_BEDROCK_BOUNDS_ONLY bit is 0x10000000 and is valid only as specified for segment-domain headers. Every PT_LOAD has p_align equal to the selected page size or a larger power-of-two alignment; the minimum supported page alignment is 4096 bytes.

3.5 Section and Symbol Policy

Standard ELF section types and the SHF_WRITE, SHF_ALLOC, SHF_EXECINSTR, and SHF_TLS flags retain their generic meanings. Symbol binding and type occupy st_info in the ordinary ELF encoding. Architecture-specific st_other bits are reserved and must be zero.

In an ET_REL object, st_value is a byte offset from the start of the defining section. After final link, it is the symbol's pre-segment virtual byte address. This transition does not convert ordinary function symbols into CS:PC pairs or ordinary object symbols into segmented descriptors.

3.6 Bedrock Object Attributes

Tag_Bedrock_Far_Model is a 32-bit architecture attribute. An absent attribute has value zero and declares no far-address contract. Value one selects the canonical far representation: a pre-segment address paired with a disabled or bounds-only segment image. Section 6 defines compatibility and rejection rules for this value.

3.7 ABI Attribute Note Encoding

Table 3-4. Bedrock ABI Attribute Note

Field	Value
section	.note.bedrock
owner name	BEDROCK
owner namesz	8 including the terminating null byte
note type	NT_BEDROCK_ABI_ATTRIBUTES
note type value	1
descriptor size bytes	8
descriptor word 0 tag	Tag_Bedrock_Far_Model
descriptor word 0 type	u32
descriptor word 1	reserved u32; must be zero

An input object may contain at most one Bedrock ABI attribute note. A linker merges compatible input values and emits exactly one note in the output.

4 PROGRAM LOADING

Bedrock ELF program loading follows the ordinary ELF segment model. The ELF ABI defines executable mapping, loader-visible object rules, and the minimum execution environment needed to begin running the image; language runtimes and OS ABIs define their own entry argument protocol above this layer.

4.1 Loading Rules

The loader may execute `ET_EXEC` and executable `ET_DYN` images. It honors `PT_INTERP`, maps each loadable segment at its declared page alignment, and rejects an alignment below 4096 bytes. For a segment whose `p_memsz` exceeds `p_filesz`, the loader fills every byte in the half-open range $[\texttt{p\_vaddr} + \texttt{p\_filesz}, \texttt{p\_vaddr} + \texttt{p\_memsz})$ with zero. Object and section alignment requirements remain binding, but the start of this gap has no additional alignment guarantee.

4.2 Page Permissions

Every `PT_LOAD` shall set `PF_R`. A loader rejects a `PT_LOAD` whose flags are zero or contain `PF_W` or `PF_X` without `PF_R`. For an accepted segment, `PF_W` and `PF_X` map independently to PTE W and X as shown below. This restriction is required because a present Bedrock page is readable and cannot implement a write-only or execute-only ELF segment.

Table 4-1. ELF Segment Permission Mapping

ELF Flags	PTE Flags	Meaning
`PF_R`	P, U	readable user page
`PF_R|PF_W`	P, U, W	readable writable user page
`PF_R|PF_X`	P, U, X	readable executable user page
`PF_R|PF_W|PF_X`	P, U, W, X	readable writable executable user page

4.3 Execution Environment

At transfer to `e_entry`, the active code context must permit fetch from the entry address, the data context must cover the mapped ordinary data regions, and the stack context must permit reads and writes to the stack provided by the platform/runtime environment ABI. This document does not fix the numeric values or descriptor images of CS, DS, or SS.

Executable images and ordinary shared objects form one ABI-visible near-call linkage domain. Their function symbols are pre-segment code addresses, not CS:PC pairs. A segment-changing call is therefore outside the ordinary dynamic linking contract; a runtime or OS ABI may expose a near-call veneer for such a transfer.

A `bedrock-elf-far` executable additionally requires an OS ABI that permits validated user updates to GS1 through GS5 and preserves GS0 through GS5 as architectural thread state across asynchronous entry and return.

4.4 Initial Non-Segment State

Immediately before transfer to the entry point, the loader clears `FLAGS`, `FSTATUS`, and `FFLAGS`. Segment-register state is governed by the execution-environment rules above rather than by this initial-value rule.

5 SECTIONS AND SYMBOLS

Bedrock uses ordinary ELF sections with the usual ELF meanings. Only the Bedrock-specific section conventions are listed below.

5.1 Bedrock-Specific Sections

Table 5-1. Bedrock-Specific Section Conventions

Section	Attributes
`.got.far`	alloc read write far pointer entries
`.note.bedrock`	note
`.bedrock.segdomains`	linker metadata

5.2 Symbol Model

Bedrock supports the standard local, global, and weak bindings and the default, hidden, and protected visibility classes. The standard notype, object, function, TLS, ifunc, section, and file symbol types retain their ELF meanings. Two processor-specific far-function types extend that set and are listed below.

- Function symbols name the first instruction of the callable entry point.
- Object symbols name the first byte of the object storage.
- In `ET_REL`, ELF `st_value` stores a byte offset relative to the symbol's defining section. In `ET_EXEC` and `ET_DYN` after final link, it stores the symbol's pre-segment virtual byte address as defined in Section 2.4.
- STT BEDROCK FAR FUNC has processor-specific type value 13 and names a canonical LCALL/LRET far-call entry.
- A near STT FUNC entry and far STT BEDROCK FAR FUNC entry for one source-level function are distinct symbols with distinct addresses.

5.3 Processor-Specific Symbol Types

Table 5-2. Bedrock ELF Symbol Types

Name	Value	Meaning
`STT_BEDROCK_FAR_FUNC`	13	canonical LCALL/LRET far-call entry
`STT_BEDROCK_FAR_IFUNC`	14	near-call resolver returning a canonical far function pair in GS1:R0

6 FAR SEGMENT DOMAINS AND LINKING

Profile: bedrock-elf-far

6.1 Profile Scope

The far profile may appear in `ET_REL` inputs and in `ET_EXEC` or `ET_DYN` output images. It applies to non-TLS data symbols and to functions explicitly typed `STT_BEDROCK_FAR_FUNC`. TLS is not assigned to a far segment domain; Section 9 defines the only TLS addressing models.

6.2 Object Compatibility

Every object that defines or consumes a far relocation, processor-specific far symbol type, `.got.far` entry, or segment-domain record must carry `Tag_Bedrock_Far_Model=1` in `.note.bedrock`. A value-zero or attribute-absent object may participate in the same link only when it neither defines nor consumes a far construct.

The linker examines all input attributes before resolving far constructs. It rejects an unknown value and rejects any far-using input without value one. If at least one input selects value one, the output also emits value one. No translated-window or alternative far-pointer representation is link-compatible with this profile.

6.3 Segment-Domain Metadata

The `.bedrock.segdomains` section is a little-endian, non-allocatable `SHT_PROGBITS` metadata section with no section flags. Its `sh_entsize` is 16 bytes, and every record has the following layout:

Table 6-1. Segment-Domain Record Layout

Offset	Field	Type	Meaning
0	`section_index`	`u32`	section header index of an allocatable non-TLS data or code section
4	`domain_id`	`u32`	nonzero object-local segment-domain identifier
8	`segment_image`	`u64`	zero in ET REL unless fixed by the producer; final validated image in ET EXEC or link-time image template in ET DYN

6.4 Final Program Header

Each output domain has exactly one `PT_BEDROCK_SEGDOM` program header. The processor-specific `p_type` value is `0x70000000`. `PT_LOAD` headers continue to describe file mapping; the domain header describes the validated segment image:

Table 6-2. PT_BEDROCK_SEGDOM Field Interpretation

Field	Interpretation
p paddr	nonzero image-local domain identifier
p vaddr	domain linear base before ET DYN load bias

Field	Interpretation
p memsz	domain byte span; linker pads the domain to an exactly representable architectural segment span
p flags	PF R, PF W, and PF X describe access; PF BEDROCK BOUNDS ONLY is mandatory
segment mode	enabled domains are bounds-only with b equal to one; translated-window domains are rejected
p align	4096 or greater
p offset and p filesz	zero; PT LOAD headers continue to describe file mapping
ET DYN rule	loader adds load bias to p vaddr, validates the resulting base and span, and constructs a bounds-only SEG(D)

6.5 Domain Rules

- Every section named by a record must be SHF ALLOC and must not be SHF TLS.
- Every final segment-domain image has nonzero mantissa and b equal to one; translated-window images are not ABI-conforming far domains.
- Data domains may contain object symbols; executable domains may contain STT BEDROCK FAR FUNC symbols and must have PF X in the final domain header.
- A domain containing writable data and executable code is rejected unless an OS ABI explicitly permits writable executable mappings.
- Domain identifiers are local to one input object; a static linker may coalesce domains only when its linker script explicitly requests the same output domain.
- The static linker assigns each output domain one final segment image and rewrites output metadata records with that image.
- The complete output domain, including the end of its last object, must lie inside the segment image bounds.
- Symbols without a segment-domain record remain in the ordinary near linkage domain and cannot be named by far-address relocations.
- Discarded input sections discard their associated segment-domain records.

6.6 Far Pointer Relocation Pair

An ELF far pointer object and a `.got.far` entry are 16-byte aligned and contain two adjacent 64-bit words:

Table 6-3. ELF Far Pointer Object and GOT Entry Layout

Offset	Field	Type	Static Relocation	Published `.got.far` Value
`+0x00`	address	`u64`	`R_BEDROCK_FAR_ADDR64`	canonical pre-segment data address or far-call entry address
`+0x08`	segment image	`u64`	`R_BEDROCK_FAR_SEGMENT64`	canonical disabled or bounds-only data image or far-call CS image

For a static far pointer object, the explicit RELA addend applies only to the address word. The segment-image relocation has addend zero. Both relocations name the same symbol and together form one indivisible link-time contract, although the loader need not update the two words atomically before publication. An address word of zero denotes a C null pointer, but neither the linker nor the loader clears the paired image word for that reason. The image relocation, canonical-image requirement, and complete-pair publication rules still apply.

6.7 Static Link Rules

- R BEDROCK FAR ADDR64 and R BEDROCK FAR SEGMENT64 must occur as a pair against the same symbol at adjacent eight-byte fields of one aligned 16-byte object.
- The linker rejects a relocation pair whose symbol is not assigned to exactly one segment domain.
- The linker rejects every far relocation whose symbol has STT TLS or resides in an SHF TLS section.
- The linker accepts an address in the half-open domain range and also accepts the exclusive limit only for a relocation denoting exactly one-past a complete STT OBJECT symbol; other out-of-range results are rejected.
- A relocatable link preserves the pair and updates section indices and object-local domain identifiers as needed.
- A final static link resolves both words and may retain .bedrock.segdomains for inspection even when no runtime processing is required.

6.8 Dynamic Link Rules

- A final ET DYN link emits one PT BEDROCK SEGDOM header per output domain and converts observable far addresses to loader-visible far dynamic relocations or .got.far entries.
- A local noninterposable far address applies R BEDROCK RELATIVE to its pre-segment address word to add ET DYN load bias and R BEDROCK FAR DOMAIN64 to its bounds-only image word.
- Interposable far data symbols use R BEDROCK FAR GLOB DAT; interposable far function symbols use R BEDROCK FAR JUMP SLOT.
- The loader constructs every domain image after load bias is known, applies complete far relocations, then publishes .got.far to application threads.
- The loader validates that every published far pair has a canonical disabled or bounds-only image; enabled addresses lie within bounds except for a relocation explicitly recognized as exactly one-past a complete object.
- Far GOT and relocation results are fully initialized before application threads can observe them and are immutable afterward; no 16-byte atomic update ABI exists.
- Unloading a shared object ends the lifetime of its data, TLS instances, near entries, far-call entries, segment images, and every pointer that designates them.

6.9 Assembler Directive

`.farptr symbol+addend` emits one aligned, zero-filled 16-byte object and the paired address and segment-image RELA relocations above. The symbol must be a non-TLS data symbol or an `STT_BEDROCK_FAR_FUNC` symbol assigned to exactly one segment domain.

7 RELOCATION SET

Bedrock ELF objects use RELA relocation entries with explicit addends. The Bedrock ABI defines how each relocation type patches instruction or data fields.

7.1 Expression Model

Every static relocation starts with the canonical expression `S+A`, where the symbol and explicit RELA addend are interpreted in the address space of the relocation family. Ordinary relocations use ABI symbol addresses, TLS relocations use GS0-relative offsets, and far relocations combine a canonical pre-segment address with a validated bounds-only domain image.

Absolute and section-relative results are permitted when the selected relocation type defines them. PC-relative address materialization subtracts `P`, the address of the patched field. Direct branch and call payloads subtract `N`, the address of the instruction following the patched instruction. A result that does not fit the signedness and width stated in the relocation table is a link error; the linker must not truncate it.

For a PC-relative address-materialization relocation, let `I` be the address of byte zero of the containing instruction and let $\delta = \mathtt{P} - \mathtt{I}$ be the relocation field's byte offset. Let A_{src} be the addend in the source expression before field-offset bias. The encoded value shall equal the target plus A_{src} minus `I`. Because the relocation-table expressions subtract `P`, the assembler records the ELF addend as $\mathtt{A} = A_{\mathrm{src}} + \delta$ for `PCREL`, `GOTPCREL`, `GOT_BASE_PCREL`, and `TLSDESC_GOTPCREL` relocations. The linker does not infer instruction boundaries. The PLT addend described in Section 8.3 is one instance of this general rule.

7.2 Expression Terms

Table 7-1. Relocation Expression Terms

Term	Meaning
`S`	Runtime byte address of the symbol named by the relocation after symbol resolution.
`A`	Explicit addend stored in the ELF64 Rela entry.
`P`	Runtime byte address of the relocation field being patched.
`N`	Runtime byte address of the next instruction after the instruction being patched.
`SB`	Runtime byte address of the beginning of the symbol's defining section.
`B`	Runtime load base address of the loaded object that owns the relocation.
`GB`	Runtime byte address of the global offset table base for the loaded object.
`GOT(S)`	Runtime byte address of the GOT slot allocated for symbol S.
`PLT(S)`	Runtime byte address of the PLT entry allocated for symbol S.
`TLS(S)`	Byte offset of TLS symbol S from the ABI TLS base in GS0.
`TLSDESC(S)`	Runtime byte address of the TLSDESC entry allocated for TLS symbol S.
`SEG(S)`	Final validated bounds-only segment register image of the segment domain containing non-TLS data or far function symbol S.
`FARADDR(S)`	Canonical pre-segment virtual address S of a non-TLS data symbol or far-call entry in its bounds-only domain.
`SEGDOM(D)`	Final validated bounds-only segment register image of image-local segment-domain identifier D.
`FAR(S)`	The ordered 128-bit pair whose low word is FARADDR(S) and whose high word is the canonical bounds-only SEG(S).

Term	Meaning
`FarResolver(X)`	The 128-bit far function pair returned in GS1:R0 by calling the resolver at near-call address X.
`Resolver(X)`	Runtime byte address returned by calling the resolver function at address X.

7.3 GOT and PLT Model

The ordinary GOT stores 64-bit addresses for position-independent code and dynamic binding. A GOT relocation allocates or references the symbol's slot in the loaded object. The PLT contains executable near-call entries; a PLT relocation names that entry rather than the final function address. Every ordinary PLT slot is resolved before application entry or constructors execute.

GOT and PLT function addresses stay inside the ABI-visible near-call linkage domain. The far GOT is a separate table of aligned 16-byte address/image pairs, is completely resolved before publication, and has no corresponding far PLT.

7.4 Relocations

Table 7-2. Bedrock ELF Relocation Types

Type	Name	Size	Calculation
0	R_BEDROCK_NONE	0	none
1	R_BEDROCK_ABS8	8-bit unsigned	S + A
2	R_BEDROCK_ABS16	16-bit unsigned	S + A
3	R_BEDROCK_ABS32S	32-bit signed	S + A
4	R_BEDROCK_ABS64	64-bit unsigned	S + A
5	R_BEDROCK_IMM8S	8-bit signed	S + A
6	R_BEDROCK_IMM16S	16-bit signed	S + A
7	R_BEDROCK_IMM32S	32-bit signed	S + A
8	R_BEDROCK_IMM64	64-bit unsigned	S + A
9	R_BEDROCK_DISP8S	8-bit signed	S + A
10	R_BEDROCK_DISP16S	16-bit signed	S + A
11	R_BEDROCK_DISP32S	32-bit signed	S + A
12	R_BEDROCK_DISP64	64-bit unsigned	S + A
13	R_BEDROCK_PCREL8S	8-bit signed	S + A - P
14	R_BEDROCK_PCREL16S	16-bit signed	S + A - P
15	R_BEDROCK_PCREL32S	32-bit signed	S + A - P
16	R_BEDROCK_PCREL64	64-bit signed	S + A - P
17	R_BEDROCK_BRDISP8S	8-bit signed	S + A - N
18	R_BEDROCK_BRDISP16S	16-bit signed	S + A - N
19	R_BEDROCK_BRDISP32S	32-bit signed	S + A - N
20	R_BEDROCK_CALL16S	16-bit signed	S + A - N
21	R_BEDROCK_CALL32S	32-bit signed	S + A - N
22	R_BEDROCK_SECTION_REL32	32-bit unsigned	S + A - SB

Type	Name	Size	Calculation
23	R_BEDROCK_SECTION_REL64	64-bit unsigned	S + A - SB
24	R_BEDROCK_GOT64	64-bit unsigned	GOT(S) + A
25	R_BEDROCK_GOTPCREL32S	32-bit signed	GOT(S) + A - P
26	R_BEDROCK_GOTPCREL64	64-bit signed	GOT(S) + A - P
27	R_BEDROCK_GOTOFF32S	32-bit signed	S + A - GB
28	R_BEDROCK_GOTOFF64	64-bit signed	S + A - GB
29	R_BEDROCK_GOT_BASE_PCREL32S	32-bit signed	GB + A - P
30	R_BEDROCK_GOT_BASE_PCREL64	64-bit signed	GB + A - P
31	R_BEDROCK_PLT16S	16-bit signed	PLT(S) + A - N
32	R_BEDROCK_PLT32S	32-bit signed	PLT(S) + A - N
33	R_BEDROCK_PLT64	64-bit unsigned	PLT(S) + A
34	R_BEDROCK_RELATIVE	64-bit unsigned	B + A
35	R_BEDROCK_GLOB_DAT	64-bit unsigned	S + A
36	R_BEDROCK_JUMP_SLOT	64-bit unsigned	S + A
37	R_BEDROCK_COPY	variable-size	copy symbol size bytes
38	R_BEDROCK_IRELATIVE	64-bit unsigned	Resolver(B + A)
39	R_BEDROCK_TLS_OFFSET32S	32-bit signed	TLS(S) + A
40	R_BEDROCK_TLS_OFFSET64	64-bit signed	TLS(S) + A
41	R_BEDROCK_TLSDESC_GOTPCREL32S	32-bit signed	TLSDESC(S) + A - P
42	R_BEDROCK_TLSDESC_GOTPCREL64	64-bit signed	TLSDESC(S) + A - P
43	R_BEDROCK_TLSDESC_CALL	0	none
44	R_BEDROCK_TLSDESC	2 x u64	write {function, argument} for S
45	R_BEDROCK_FAR_ADDR64	64-bit unsigned	FARADDR(S) + A
46	R_BEDROCK_FAR_SEGMENT64	64-bit unsigned	SEG(S)
47	R_BEDROCK_FAR_DOMAIN64	64-bit unsigned	SEGDOM(A)
48	R_BEDROCK_FAR_GLOB_DAT	128-bit unsigned	{FARADDR(S) + A, SEG(S)}
49	R_BEDROCK_FAR_JUMP_SLOT	128-bit unsigned	FAR(S)
50	R_BEDROCK_FAR_IRELATIVE	128-bit unsigned	FarResolver(B + A)

7.5 Relocation Families and Additional Constraints

Relocations 1–12 write absolute data, immediate, or effective-address payloads. Relocations 13–23 write PC-relative, next-instruction-relative, or section-relative results. Relocations 24–33 address GOT slots, the GOT base, symbols relative to that base, and PLT entries. Relocations 34–38 are ordinary dynamic-loader operations. Relocations 39–44 implement GS0 local-exec and TLSDESC TLS. Relocations 45–50 construct or resolve canonical far pairs. The table defines widths and calculations; Sections 6, 8, and 9 supply additional rules.

8 DYNAMIC LINKING

Dynamic objects use a fixed eager-binding Bedrock GOT and PLT contract. Lazy binding and a runtime PLT resolver protocol are not part of the current draft ELF ABI.

8.1 Dynamic Linking Rules

Ordinary dynamic linking supports standard and indirect functions, copy relocations, and `R_BEDROCK_IRELATIVE`. The loader resolves every ordinary `R_BEDROCK_JUMP_SLOT` and `R_BEDROCK_IRELATIVE` relocation before transferring control to the program entry point or any constructor. Shared-object calls use near `CALL/RET` within the ABI-visible linkage domain. The GOT base symbol is `_GLOBAL_OFFSET_TABLE_`.

There is no `PLT0`. Every ordinary PLT entry occupies 32 bytes and has 16-byte alignment. Far binding uses the same eager publication rule: every `.got.far` entry is a 16-byte aligned 16-byte pair, is bound eagerly, and is immutable after publication. There is no far PLT. An external far call loads the final pair from `.got.far` and transfers with `LCALL`.

8.2 GOT Reserved Entries

Table 8-1. Global Offset Table Reserved Entries

Index	Name	Meaning
0	`_DYNAMIC`	dynamic section address
1	reserved	zero; no loader-handle meaning
2	reserved	zero; no lazy-resolver meaning

8.3 PLT Layout

Table 8-2. Procedure Linkage Table Layout

Offset	Bytes or field	Effect
`+0x00`	`e7 c9 80 67`	`JMP.Q [PC + disp64]`
`+0x04`	`disp64`	little-endian `R_BEDROCK_GOTPCREL64` relocation field
`+0x0c`	twenty `01` bytes	canonical `NOP` padding through offset `+0x1f`

A PLT relocation uses explicit addend `+4`. Architectural PC names byte zero of the current instruction, whereas relocation place `P` is the field at entry offset four; therefore `GOT(S)+A-P` evaluates to `GOT(S)-PLTentry`, the displacement consumed by the PC-relative EA. The referenced `.got.plt` slot is 8-byte aligned and contains the eagerly resolved 64-bit near target. Both the PLT entry and its slot must be reachable under the active CS bounds used by that PC-relative EA.

A PLT transfer preserves `R0` through `R7`, `F0` through `F7`, `GS1` through `GS5`, `FLAGS`, and all segment state. The loader writes each final `.got.plt` value before publication. Published slots are immutable and may be covered by `PT_GNU_RELRO`; no post-publication rebinding is permitted. A later lazy-binding facility requires a distinct ELF profile and resolver ABI.

8.4 Far Binding Rules

- The loader resolves every R BEDROCK FAR GLOB DAT, R BEDROCK FAR JUMP SLOT, and R BEDROCK FAR IRELATIVE relocation before application code can observe the entry.
- A far function GOT entry contains the canonical STT BEDROCK FAR FUNC address and is called with LCALL.
- No lazy far PLT state exists; a far-call resolver never consumes or overwrites C argument registers at the call site.
- Interposition resolves and writes the complete 16-byte pair before loader publication; the ABI requires no 16-byte atomic store.
- STT BEDROCK FAR IFUNC resolvers are called with the normal near C ABI and return a pre-segment code address in R0 and a canonical disabled or bounds-only CS image in GS1.
- The loader rejects a far resolver result whose image is noncanonical or whose non-null function address lies outside an enabled image's bounds.
- Published .got.far entries are immutable; post-publication rebinding or mutation is forbidden.

9 THREAD-LOCAL STORAGE

Thread-local storage is addressed through the GS0 segment register. TLS references that remain inside the program and have a link-time constant offset use the local-exec model. TLS references that cross dynamic object boundaries, may be interposed, or otherwise require runtime resolution use the TLSDESC model.

9.1 TLS Base Model

GS0 is the TLS base register. A TLS offset is translated by GS0 segment pre-translation before page-table translation. The local-exec model encodes a link-time constant byte offset from the GS0 base; TLSDESC supplies runtime resolution when the reference crosses a dynamic-object boundary or remains interposable.

Table 9-1. TLS Relocation Families

Model	Relocations
local-exec	`R_BEDROCK_TLS_OFFSET32S`
	`R_BEDROCK_TLS_OFFSET64`
TLSDESC	`R_BEDROCK_TLSDESC_GOTPCREL32S`
	`R_BEDROCK_TLSDESC_GOTPCREL64`
	`R_BEDROCK_TLSDESC_CALL`
	`R_BEDROCK_TLSDESC`

Far TLS relocations and far TLS static initializers are forbidden. Taking the address of a TLS object linearizes its GS0-relative address at runtime and returns an ordinary pre-segment pointer to that thread instance. Explicit far conversion operates on that already materialized pointer and retains neither a GS0 image nor a TLS offset. Copying the pointer to another thread does not retarget it; the pointer becomes invalid when its originating thread terminates or the defining shared object is unloaded.

9.2 Model Selection

- A TLS symbol defined in the same program image may use local-exec addressing when its byte offset from GS0 is link-time constant.
- A TLS reference from a shared object or to an interposable TLS symbol uses TLSDESC.
- The linker may relax a TLSDESC sequence to local-exec when symbol binding and final placement make the GS0-relative offset constant.
- A source expression `S+k` uses the descriptor for `S`; after resolution, the caller adds `k` to the returned offset. The descriptor relocations never encode the subobject addend `k`.

9.3 TLSDESC Entry

Table 9-2. TLSDESC Entry Layout

Offset	Field	Type	Meaning
+0x00	function	u64	near-call address of the resolver or fast function
+0x08	argument	u64	signed direct GS0-relative offset or resolver-specific opaque token

Each entry occupies 16 bytes in .got and has 16-byte alignment. The loader resolves the symbol binding, writes both words, and completes any memory ordering required to publish the pair before application code can observe it. Both words are immutable after publication and may subsequently be protected by PT_GNU_RELRO. The ABI requires no public name for either the resolver or fast function; their near-call addresses may be loader-private.

The argument word belongs to the implementation of the selected function. A fast function may load [R0+8] and return it directly. An opaque token is not application data and remains valid only while the object that defines the TLS symbol is loaded. A resolver may perform dynamic per-thread lookup or allocation, but neither it nor the fast function may rewrite or self-patch the published descriptor. This per-thread work is not lazy symbol binding: the loader has already resolved the TLS symbol and published the final immutable pair.

9.4 TLSDESC Call ABI

The canonical sequence is exactly:

```
LEA.Q [PC + descriptor@TLSDESC_GOTPCREL], R0
CALL  [R0]
```

The LEA.Q places the descriptor address in R0. CALL [R0] reads the 64-bit near-call target from descriptor offset zero and does not replace R0, so the called function receives the descriptor address in R0. No intervening move is part of the canonical sequence.

The function returns a valid signed byte offset from the GS0 TLS base in R0. It preserves GS0 and every register and state item that the normal C ABI designates callee-saved, and it may clobber normal caller-saved state. There is no recoverable failure return or reserved offset: zero is a valid TLS offset. A resolver that cannot produce an offset follows a platform-defined fatal path and does not return.

9.5 TLSDESC Relocations

Descriptor address. R_BEDROCK_TLSDESC_GOTPCREL32S and R_BEDROCK_TLSDESC_GOTPCREL64 apply to the PC-relative displacement field of the canonical LEA.Q and compute the address of the descriptor for STT_TLS symbol S. The source expression has semantic TLS addend zero. Under the PC-relative field-bias rule in Section 7.1, the RELA addend is exactly $A = \delta$; it contains the instruction-field offset and no TLS subobject addend.

Call marker. `R_BEDROCK_TLSDESC_CALL` has size zero, changes no byte, and names the same TLS symbol as the paired TLSDESC GOTPCREL relocation on the immediately preceding `LEA.Q`. Its addend is zero and its place is byte zero of the exact immediately following `CALL [R0]`. It is a static-link relaxation marker, is consumed by the static linker, and is never emitted as a dynamic relocation. A producer shall not place a label or control-flow target between the two instructions or at a nonzero byte of either instruction when emitting a relaxable sequence.

Descriptor initialization. `R_BEDROCK_TLSDESC` applies only to `STT_TLS` symbols, has addend zero, and is placed at offset zero of an aligned descriptor. It directs the loader to initialize two adjacent 64-bit words with the selected `{function, argument}` pair for `S`. It is not an integer calculation. The link or load that would leave a weak TLS symbol unresolved shall reject the object: GS0-relative offset zero is valid and cannot also encode absent weak binding. A weak symbol that resolves to a definition is handled normally.

The linker may merge descriptors only when they name the same resolved binding `S` and have the required zero semantic addend. If any unrelaxed use remains, a final dynamic link emits one aligned pair in `.got` and the corresponding dynamic `R_BEDROCK_TLSDESC`. The loader initializes the pair eagerly before publication. A final executable with no runtime loader shall relax every TLSDESC use; otherwise the static link fails. If all uses are relaxed, the final link discards the descriptor and its initialization relocation.

9.6 TLSDESC Local-Exec Relaxation

Relaxation is permitted only after final binding proves that `S` is non-preemptible and `TLS(S)` is a link-time constant. It replaces the entire canonical pair without changing the section size or the address of any following instruction:

- With `R_BEDROCK_TLSDESC_GOTPCREL32S`, the original 7-byte `LEA.Q` and 4-byte `CALL [R0]` occupy 11 bytes. If `TLS(S)` fits signed 32 bits, the linker emits `LEN 11, MOV.Q TLS(S), R0`, applies `R_BEDROCK_TLS_OFFSET32S` with addend zero to its 32-bit immediate payload, and writes four zero padding bytes after the natural 7-byte form.
- With `R_BEDROCK_TLSDESC_GOTPCREL64`, the original 11-byte `LEA.Q` and 4-byte `CALL [R0]` occupy 15 bytes. The linker emits `LEN 15, MOV.Q TLS(S), R0`, applies `R_BEDROCK_TLS_OFFSET64` with addend zero to its 64-bit immediate payload, and writes four zero padding bytes after the natural 11-byte form.

The linker consumes both code relocations. The removed call may eliminate normal caller-saved clobbers and unobservable resolver stack activity; code may not depend on either. If no unrelaxed reference remains, no descriptor is allocated.

10 CODE MODELS

Code models define placement and locality assumptions that assemblers, linkers, and compilers may use when selecting address materialization, displacement, GOT, PLT, and relocation forms. Bedrock 32-bit displacement and direct control-transfer forms are signed, so direct 32-bit relative references cover a -2 GiB to +2 GiB - 1 window.

Table 10-1. Bedrock Code Models

Model	Placement contract	Default access strategy
`low`	static addresses in `0x00000000..0x7fffffff`	signed 32-bit absolute data and direct transfers
`small`	referenced code, data, GOT, and PLT fit one signed 32-bit relative window	PC-relative access; GOT/PLT for interposable symbols
`medium`	code, GOT, and PLT fit one signed 32-bit window; data placement is unrestricted	direct code; GOT-resolved 64-bit data addresses
`high`	static high image with internal references in a signed 32-bit displacement window	image-base or PC-relative access with full-width fallback
`large`	unrestricted 64-bit address space	full-width materialization and indirect transfer

10.1 low Model Details

The `low` model is a static image model whose named code and data lie between `0x00000000` and `0x7fffffff`. Code uses direct signed 32-bit branch or call payloads; data uses absolute or base-relative signed 32-bit payloads. A GOT or PLT is not required by the model.

Its default relocation set is `R_BEDROCK_ABS32S`, `R_BEDROCK_IMM32S`, `R_BEDROCK_DISP32S`, `R_BEDROCK_BRDISP32S`, and `R_BEDROCK_CALL32S`. The linker may shrink a direct transfer or address materialization after final placement proves that the result fits a shorter encoding.

10.2 small Model Details

The `small` model is position-independent and assumes that local code, nearby data, the GOT, and the PLT fit the signed 32-bit relative window of the use site or selected base. Code uses direct signed 32-bit transfers; data uses PC-relative or GOTPCREL signed 32-bit materialization.

Its default relocations are `R_BEDROCK_BRDISP32S`, `R_BEDROCK_CALL32S`, `R_BEDROCK_GOTPCREL32S`, `R_BEDROCK_PLT32S`, and `R_BEDROCK_TLSDESC_GOTPCREL32S`. The linker may shrink a relative form or replace GOT/TLSDESC access with a direct form when the resolved symbol is local and in range.

10.3 medium Model Details

The `medium` model keeps code, GOT, and PLT within a signed 32-bit relative window but permits data objects anywhere in the 64-bit address space. Code uses direct signed 32-bit transfers. Data normally uses a GOT-resolved 64-bit address reached through a signed 32-bit GOT reference.

Its default relocations are `R_BEDROCK_BRDISP32S`, `R_BEDROCK_CALL32S`, `R_BEDROCK_GOTPCREL32S`, `R_BEDROCK_GOT64`, `R_BEDROCK_PLT32S`, and `R_BEDROCK_TLSDESC_GOTPCREL32S`. A linker may replace a GOT data reference with a direct relative reference when final placement puts the object in range.

10.4 high Model Details

The `high` model is a static high-address image. Internal code may use direct signed 32-bit transfers or image-base-relative addressing. Data uses a PC-relative GOT or an image-base-relative form when in range; the GOT and PLT are image-local or base-relative.

Its default relocations are `R_BEDROCK_BRDISP32S`, `R_BEDROCK_CALL32S`, `R_BEDROCK_GOTPCREL32S`, `R_BEDROCK_ABS64`, and `R_BEDROCK_DISP64`. Compact internal references are permitted when final placement proves the signed 32-bit range; full-width and base-relative sequences remain conforming.

10.5 large Model Details

The `large` model makes no placement assumption. Code uses full-width address materialization followed by an indirect transfer when necessary; data, GOT, and PLT addresses are full width.

Its default relocations are `R_BEDROCK_ABS64`, `R_BEDROCK_PCREL64`, `R_BEDROCK_GOTPCREL64`, `R_BEDROCK_PLT64`, and `R_BEDROCK_TLSDESC_GOTPCREL64`. Final placement may justify relaxing any full-width form to a shorter encoding.

11 ASSEMBLER CONTRACT

Default ABI: bedrock-elf

11.1 Strict Mode

- Pure ISA syntax is accepted.
- Ambiguous symbolic operands require an explicit relocation annotation.
- Noncanonical encodings are rejected.
- The .farptr directive is accepted only when bedrock-elf-far is enabled and emits the required paired RELA relocations.
- Emitting a far relocation, far symbol type, .got.far entry, or segment-domain record also emits Tag Bedrock Far Model value 1; suppressing the required attribute is an error.

11.2 Relaxation

- Branch and call payload widths may be selected automatically when the target is known.
- Address materialization may shrink from 64-bit to 32-bit, 16-bit, or 8-bit payloads when the relocation result fits.
- Overlong encodings are emitted only when explicitly requested.